

Informatica di Base - Seconda prova intermedia

Matricola: _____

REMINDER DI SINTASSI

Tipi di dati

Tipo	Esempio
int	42, -3
float	3.14, -0.5
str	"ciao", 'abc'
bool	True, False
list	[1, 2, 3], []

Operatori

Operatore	Significato
+ - * / // % **	Aritmetici
== != < > <= >=	Confronto
and or not	Logici
s[i] s[-i] s[a:b]	Selezione di una posizione
len(s)	Lunghezza di un iterabile
lista.append(x)	Aggiunta di un elemento alla lista

Istruzione if

```
if condizione:  
    # istruzione
```

```
if condizione:  
    # istruzione  
else:  
    # istruzione
```

```
if condizione_1:  
    # istruzione  
elif condizione_2:  
    # istruzione  
else:  
    # istruzione
```

Ciclo while

```
while condizione:  
    # istruzione
```

Funzione pura

```
def nome_funzione(parametro1, parametro2):  
    # corpo della funzione  
    return risultato  
  
x = nome_funzione(arg1, arg2)
```

Funzione void

```
def nome_funzione(parametro1, parametro2):  
    # corpo della funzione  
  
nome_funzione(arg1, arg2)
```

Esercizi

Esercizio n. 1 — 4 punti

Spiega la differenza semantica tra la "definizione" di una funzione e la sua "chiamata" (o invocazione). Cosa cambia tra funzioni pure (o produttive) e funzioni void?

Rispondi in massimo 5 righe.

Esercizio n. 2 — 2 punti

Se scriviamo `a = [1, 2]` poi `b = [1, 2]`, cosa succede se eseguiamo `b.append(3)` ?

- ☐ Solo la lista b conterrà il numero 3.
- ☐ Viene creata automaticamente una copia di a prima della modifica.
- ☐ Entrambe le variabili a e b punteranno alla lista aggiornata .
- ☐ Python genera un errore perché non è possibile avere due nomi per lo stesso oggetto.

Esercizio n. 3 — 2 punti

Quale dei seguenti tipi di dati è considerato "immutabile" in Python?

- ☐ List ☐ Int ☐ Entrambi i tipi menzionati ☐ Nessuno dei tipi menzionati

Esercizio n. 4 — 2 punti

Cosa stampa il seguente codice?

```
def f(x):  
    x = x + 1  
  
a = 5  
f(a)  
print(a)
```

- ☐ 5
- ☐ 6
- ☐ None
- ☐ Python genera un errore perché `x` non è definita fuori da `f`

Esercizio n. 5 — 6 punti

Considera il seguente codice sorgente e rispondi alle domande che seguono

Il seguente programma vuole costruire la lista che, per ogni elemento di `lista`, conta quanti elementi **maggiori di lui** lo **precedono** nella lista.

```
lista = [3, 1, 4, 2]  
  
contatori = []  
i = 0  
while i < len(lista):  
    contatore = 0  
    j = 0  
    while j < len(lista):  
        if lista[j] > lista[i]:  
            contatore = contatore + 1  
        j = j + 1  
    contatori.append(contatore)  
    i = i + 1  
  
print(contatori)
```

- ☐ Il programma termina senza errori?
- ☐ L'output è quello atteso? Se c'è un errore logico, spiegalo e scrivi il codice corretto.

Esercizio n. 6 — 4 punti

Completa la tabella di tracciamento dello stato per il seguente frammento di codice

```
def azzera(lista):  
    i = 0  
    c = 0  
    while i < len(lista):  
        if lista[i] < 0:  
            c = c + 1  
            lista[i] = 0  
            i = i + 1  
  
    lista.append(c)  
  
numeri = [5, 10, -1]  
azzera(numeri)  
print(numeri)
```

Passo	Codice sorgente	Stato - globale	Stato - funzione	Output

Esercizio n. 7 — 6 punti

Scrivi una funzione pura chiamata `filtra_lunghe(lista, n)` che riceve una lista di stringhe e un intero `n` e restituisce una nuova lista contenente solo le stringhe della lista originale che hanno più di `n` caratteri, senza modificare la lista di partenza.

Esempio: Input: `["ciao", "sì", "informatica", "no"] -- 3`

Output:

```
["ciao", "informatica"]
```

Esercizio n. 8 — 10 punti

Scrivi una funzione pura chiamata `intercala(lista, k)` che riceve una lista di interi e un intero `k` e restituisce una nuova lista in cui dopo ogni elemento della lista originale vengono inseriti `k` zeri, senza modificare la lista di partenza.

Esempio: `intercala([1, 2, 3], 2) → [1, 0, 0, 2, 0, 0, 3, 0, 0]`

Poi scrivi un programma che legge un intero `k` dall'input controllando che sia maggiore di zero (validazione dell'input), legge 4 numeri interi e li salva in una lista, e stampa il risultato di `intercala` chiamata con `k` e poi con `k + 1`.

Esempio con `k = 1` e lista `[5, 3, 8, 1]`:

Output:

```
[5, 0, 3, 0, 8, 0, 1, 0]
[5, 0, 0, 3, 0, 0, 8, 0, 0, 1, 0, 0]
```